

Project Overview

Over de komende zes middagen bouw je je eigen lightweight ORM framework in Java. Je begint met kale JDBC en bouwt daar stap voor stap abstracties bovenop: connection management, object mapping, transacties, polymorfisme, entity relaties, fetch strategies en uiteindelijk caching en performance-optimalisaties. Aan het eind van de rit heb je een mini-ORM die je kunt vergelijken met wat Hibernate onder de motorkap doet.

Je gaat de ORM toepassen op een voorbeeldproject. Het domein voor het voorbeeldproject is een **onderwijsadministratie**. Je werkt met entiteiten als [Student](#), [Course](#), [Enrollment](#), [Teacher](#) en [Department](#).

De H2 file-based database die je in de ochtendlabs hebt gebruikt, is wat je ook in de middag mag gebruiken. Wil je liever iets anders? Dat kan, overleg even met de trainer(s).

Middag 1: De Fundering – Connection Management & Object Mapping

Goals

De ochtend stond in het teken van JDBC: connecties openen, SQL uitvoeren, ResultSets verwerken. Nu ga je die kennis gebruiken om het begin te leggen van je eigen ORM framework. Je bouwt een herbruikbare laag die de repetitieve JDBC-boilerplate wegabstraheert. Aan het eind van dit deel kun je met een paar regels code objecten opslaan en ophalen uit een database, zonder dat de aanroepende code ook maar één `ResultSet` hoeft te zien.

Dit is wat frameworks als Hibernate in hun kern doen.

Learning Goals

- Een `ConnectionManager` implementeren die connecties beheert via een `DataSource` en veilig opent en sluit
- Een generiek object-mapping mechanisme bouwen dat `ResultSet`s automatisch mapt naar Java-objecten op basis van annotaties of conventie
- Beoordelen welke trade-offs je maakt bij het ontwerpen van een mapping-strategie (annotaties vs. naamconventies vs. handmatige configuratie)

Part 1: ConnectionManager bouwen

Bouw een `ConnectionManager` klasse die verantwoordelijk is voor het beheren van database-connecties. Deze klasse ontvangt een `DataSource` en biedt methodes om connecties op te halen en netjes te sluiten. Denk ook alvast na over wat er moet gebeuren als het openen van een connectie mislukt.

Maak daarnaast een configuratieklasse die de H2 `DataSource` aanmaakt, vergelijkbaar met wat je in de ochtendlab hebt gedaan, maar nu als onderdeel van je framework.

Success criteria:

- De `ConnectionManager` kan connecties uitdelen aan aanroepende code
- Connecties worden correct gesloten (geen resource leaks)
- Een unit test bewijst dat een connectie succesvol geopend en gesloten kan worden
- Basis foutafhandeling is aanwezig: als de database onbereikbaar is, krijg je een duidelijke foutmelding

Part 2: Mapping strategie implementeren

In dit deel ga je zorgen dat Java-objecten automatisch gemapt kunnen worden. Hier zijn verschillende strategieën voor. Je kunt bijvoorbeeld annotaties definiëren waarmee je Java-classes kunt markeren als database-entiteiten. Denk aan iets als `@Entity`, `@Table`, `@Column` en `@Id`. Dan ben je er nog niet, vervolgens zul je iets van een `EntityMetadata` klasse moeten bouwen die via reflectie de annotaties van een klasse uitleest en de

mapping-informatie vastlegt: welke tabel hoort bij deze klasse? Welke velden horen bij welke kolommen? Wat is het ID-veld?

Je kunt ook kiezen voor mapping middels bijvoorbeeld configuratie of naamconventies.

Pas deze mapping toe op je `Student` klasse uit de ochtendlabs.

Success criteria:

- Mapping strategie geïmplementeerd
- De mapping strategie is toegepast op de `Student` klasse
- Een test verifieert dat de metadata van `Student` correct wordt uitgelezen

Part 3: Generieke CRUD Dao

Bouw een `GenericDao<T>` die de metadata uit Part 2 gebruikt om automatisch SQL te genereren en uit te voeren voor CRUD-operaties. De repository moet `findAll()`, `findById()`, `save()`, `update()` en `delete()` ondersteunen voor elke gemapte klasse.

Test je repository met de `Student` entiteit. Voeg ook een `Course` entiteit toe (met velden als `id`, `name`, `credits`) om te bewijzen dat je repository daadwerkelijk generiek is.

Success criteria:

- `GenericDao<Student>` kan studenten opslaan, ophalen, updaten en verwijderen
- `GenericDao<Course>` werkt zonder aanpassingen aan de repository-code
- SQL wordt dynamisch gegenereerd
- Alle operaties zijn getest met de H2 database

Bonus Challenge (Optional)

Voeg een simpele query builder toe waarmee je iets kunt doen als:

```
repository.where("age", ">", 20).and("name", "LIKE", "%Jan%").findAll();
```

Dit dwingt je om na te denken over hoe je SQL veilig opbouwt (`PreparedStatement`!) en hoe je een fluent API ontwerpt. Kijk kritisch naar je eigen ontwerp: waar wordt het rommelig? Waar zou je dingen anders doen als je opnieuw begon?

Middag 2: Transacties – Commits, Rollbacks en Concurrency

Goal

Vanmorgen heb je geleerd hoe transacties werken in JDBC: autocommit uitzetten, handmatig committen of rollbacken, en wat ACID-eigenschappen in de praktijk betekenen. Nu integreer je die kennis in je framework.

Je voegt transaction management toe aan je framework zodat meerdere database-operaties als één atomic eenheid uitgevoerd kunnen worden. En je gaat ontdekken wat er gebeurt als twee threads tegelijk dezelfde data proberen te wijzigen.

Learning Goals

- Een `TransactionManager` implementeren die transacties beheert (begin, commit, rollback) bovenop de bestaande `ConnectionManager`
- De `GenericRepository` aanpassen zodat operaties binnen een transactie uitgevoerd kunnen worden
- Concurrent access simuleren en analyseren welke problemen optreden zonder juiste transactie-isolatie
- Evalueren hoe optimistic locking een alternatief biedt voor database-level locks

Project Afternoon Parts

Het project van deze middag bevat **3 delen**.

Part 1: TransactionManager

Bouw een `TransactionManager` die transacties beheert. De manager moet het mogelijk maken om een transactie te starten, te committen en te rollbacken. Binnen een transactie moeten alle operaties dezelfde connectie delen.

Een veelgebruikt patroon hiervoor is om de huidige connectie op te slaan in een `ThreadLocal`, zodat alle code binnen dezelfde thread automatisch dezelfde connectie (en dus dezelfde transactie) gebruikt.

Success criteria:

- `TransactionManager` biedt `begin()`, `commit()` en `rollback()` methodes
- Binnen een transactie wordt dezelfde connectie hergebruikt
- Na een commit of rollback wordt de connectie vrijgegeven
- Een test toont aan dat twee opeenvolgende `save()`-operaties in één transactie beide doorgevoerd worden bij commit
- Een test toont aan dat na een rollback geen van de operaties zichtbaar is in de database

Part 2: Transacties in de Repository

Pas je `GenericRepository` aan zodat deze de `TransactionManager` gebruikt. De repository moet automatisch de connectie ophalen bij de `TransactionManager` als er een actieve transactie is, en anders een losse connectie gebruiken (auto-commit gedrag).

Bouw een handige manier om een transactioneel blok te definiëren, bijvoorbeeld via een `executeInTransaction(Runnable)` patroon of iets vergelijkbaars.

Success criteria:

- Repository-operaties binnen een transactie gebruiken de transactie-connectie
- Repository-operaties buiten een transactie werken nog steeds (auto-commit)
- Een transactie die een `save()` voor `Student` en een `save()` voor `Course` combineert, wordt als geheel gecommit of gerolbackt
- Er is een test die bewijst dat een fout halverwege de transactie leidt tot een rollback van alle operaties

Part 3: Concurrency Testen

Schrijf een testscenario met meerdere threads die tegelijk dezelfde data proberen te lezen en schrijven. Gebruik het `Student` domein: twee threads proberen tegelijk de leeftijd van dezelfde student te updaten. Observeer wat er gebeurt.

Implementeer vervolgens een simpele vorm van optimistic locking door een `version` kolom toe te voegen aan je entiteiten. Bij een update controleer je of de versie nog overeenkomt met wat je eerder las. Zo niet, dan gooi je een exceptie.

Success criteria:

- Een test laat zien dat zonder locking, concurrent updates tot data verlies kunnen leiden (lost update)
- Na het toevoegen van een `version` kolom detecteert je framework conflicterende updates
- Bij een versie-conflict wordt een duidelijke exceptie gegooit (bijv. `OptimisticLockException`)
- Een test bewijst dat het optimistic locking mechanisme werkt

Bonus Challenge (Optional)

Voeg logging toe aan je `TransactionManager` die precies laat zien wanneer transacties starten, committen en rollbacken, inclusief welke SQL-statements er binnen de transactie zijn uitgevoerd. Dit is ontzettend waardevol voor debugging. Denk na over het logniveau: wat wil je altijd zien, en wat alleen bij debuggen?

Afternoon 3: Batch Operations & Polymorfisme

Goal

Vanochtend heb je gezien hoe Hibernate werkt en waar het soms wat onhandig wordt: polymorfisme, proxy-objecten en de grenzen van abstractie. Nu ga je twee dingen toevoegen aan je framework: batch operaties voor betere performance, en ondersteuning voor overerving in je entity model.

Dat laatste is uitdagend. Hoe sla je een lijst objecten op die allemaal van hetzelfde supertype zijn, maar verschillende subtypes?

Learning Goals

- Batch insert/update operaties implementeren met JDBC `addBatch()` / `executeBatch()` om de performance bij bulk-operaties te verbeteren
- Een overerving-strategie implementeren (single table inheritance) die supertype en subtypes correct mapt naar de database
- Evalueren waar de complexiteit van type-mapping toeneemt
- Vergelijken hoe je eigen framework omgaat met polymorfisme versus wat je van Hibernate hebt gezien

Project Afternoon Parts

Het project van deze middag bevat **3 delen**.

Part 1: Batch Operations

Voeg batch support toe aan je `GenericRepository`. In plaats van elke `INSERT` of `UPDATE` apart naar de database te sturen, verzamel je ze en stuur je ze in 1 keer. JDBC biedt hiervoor `addBatch()` en `executeBatch()` op `PreparedStatement`.

Bouw een `saveAll(List<T>)` en `updateAll(List<T>)` methode die batch operaties gebruiken. Meet het verschil in performance door 1000 studenten zowel één-voor-één als in batch in te voegen.

Success Criteria:

- `saveAll()` en `updateAll()` gebruiken JDBC batch operaties
- Een performancetest toont meetbaar verschil tussen één-voor-één en batch inserts
- Batch operaties werken correct binnen een transactie
- Bij een fout in de batch wordt de hele transactie terug gerold

Part 2: Single Table Inheritance

Voeg ondersteuning toe voor overerving. Gebruik de "Single Table Inheritance" strategie: alle subtypes worden opgeslagen in dezelfde tabel, met een discriminator kolom die aangeeft welk type het is.

Maak een voorbeeld met een `Person` superklasse en subtypes `Student` en `Teacher`. De tabel `persons` bevat alle kolommen voor beide types, plus een `person_type` kolom. Je framework moet bij het ophalen van data het juiste subtype instantiëren op basis van de discriminator.

Definieer hiervoor een `@Inheritance` en `@DiscriminatorColumn` annotatie (of vergelijkbaar).

Success criteria:

- `Student` en `Teacher` erven van `Person`
- Beide types worden opgeslagen in dezelfde `persons` tabel
- De discriminator kolom bepaalt welk type wordt geïnstantieerd bij ophalen
- `findAll()` op de `Person` repository levert een mix van `Student` en `Teacher` objecten op, elk met het juiste type
- Subtype-specifieke velden (bijv. `studentNumber` voor `Student`, `department` voor `Teacher`) worden correct gemapt

Part 3: Grenzen Verkennen

Test de grenzen van je framework. Probeer de volgende scenario's en documenteer wat er gebeurt (je hoeft dit dus niet op te lossen, maar dat zou wel helemaal cool zijn):

1. Haal een lijst van `Person` objecten op en probeer ze te casten naar het juiste subtype. Waar gaat dit fout?
2. Voeg een derde subtype toe (bijv. `Admin`). Hoeveel moet je aanpassen in je framework vs. in de applicatiecode?
3. Wat gebeurt er als een subtype een veld heeft dat `null` is voor andere subtypes? Hoe ga je daarmee om?

Schrijf je bevindingen op in een kort document of voeg ze toe aan de README. Vergelijk je ervaring met wat je vanochtend over Hibernate hebt geleerd. Waar zie je parallellen?

Success criteria:

- Je hebt alle drie de scenario's uitgeprobeerd en gedocumenteerd
- Je document beschrijft concreet welke problemen je tegenkwam
- Je kunt uitleggen waarom polymorfisme in een relationele database complex is
- Je trekt een vergelijking met Hibernate's aanpak

Bonus Challenge (Optional)

Implementeer een tweede overerving-strategie: "Table per Class". Elk subtype krijgt zijn eigen tabel met alle kolommen (inclusief die van het supertype). Vergelijk de voor- en nadelen met Single Table Inheritance. In welke situatie zou je welke strategie kiezen? Denk na over storage-efficiëntie, query-complexiteit en de impact op je framework-code.

Middag 4: Entity Relaties & Fetch Strategies

Goal

Tot nu toe leefden je entiteiten in isolatie: elke `Student` stond op zichzelf, elke `Course` ook. Maar in de echte wereld hangen dingen samen. Een student volgt cursussen. Een cursus wordt gegeven door een docent. Een docent hoort bij een afdeling.

Vanmorgen heb je geleerd over entity relaties, fetch strategies en het decorator pattern. Nu ga je relaties toevoegen aan je framework. En je gaat ontdekken dat de keuze tussen eager en lazy fetching niet triviaal is, dat het decorator pattern hier goed van pas komt, en dat `equals()` en `hashCode()` ineens heel belangrijk worden.

Learning Goals

- One-to-many en many-to-one relaties implementeren in het eigen ORM framework met behulp van annotaties en JOIN queries
- Een fetch strategie implementeren (eager loading) die gerelateerde entiteiten automatisch meeneemt
- `equals()` en `hashCode()` correct implementeren voor entiteiten met relaties
- Evalueren hoe fetch strategies de hoeveelheid queries en de performance beïnvloeden

Project Afternoon Parts

Het project van deze middag bevat **3 delen**.

Part 1: Relatie-annotaties en Metadata

Breid je annotatie-systeem uit met relatie-annotaties: `@OneToMany`, `@ManyToOne`, en eventueel `@JoinColumn`. Je `EntityMetadata` moet nu ook relatie-informatie kunnen uitlezen: welk veld refereert naar een andere entiteit? Wat is de foreign key kolom?

Modelleer de volgende relaties:

- Een `Department` heeft meerdere `Teachers` (one-to-many)
- Een `Teacher` hoort bij één `Department` (many-to-one)
- Een `Course` heeft één `Teacher` (many-to-one)

Success criteria:

- Relatie-annotaties zijn gedefinieerd en toegepast op de entiteiten
- `EntityMetadata` kan relatie-informatie extraheren (gerelateerde entiteit, foreign key kolom, type relatie)
- Een test verifieert dat de relatie-metadata correct wordt uitgelezen voor `Teacher` en `Department`

Part 2: Eager Fetching Implementeren

Pas je `GenericRepository` aan zodat bij het ophalen van een entiteit de gerelateerde entiteiten automatisch worden meegeladen (eager fetching). Als je een `Teacher` ophaalt, moet het bijbehorende `Department` object er ook zijn. Als je een `Department` ophaalt, moeten alle bijbehorende `Teacher` objecten gevuld zijn.

Gebruik JOIN queries om dit efficient te doen in plaats van aparte queries per relatie.

Success criteria:

- Het ophalen van een `Teacher` vult automatisch het `Department` veld
- Het ophalen van een `Department` vult automatisch de lijst van `Teachers`
- De repository gebruikt JOIN queries (niet N aparte queries)
- Geneste relaties werken: een `Course` ophalen geeft de `Teacher` met diens `Department`
- `equals()` en `hashCode()` zijn correct geïmplementeerd en werken met relaties (geen infinite loops)

Part 3: UML Documentatie

Maak een UML-klassediagram van je huidige domeinmodel. Toon alle entiteiten (`Student`, `Course`, `Teacher`, `Department`, `Person`) met hun velden en relaties. Gebruik de juiste UML-notatie voor relaties (associatie, compositie, overerving) en multipliciteit.

Je hoeft hier geen tool voor te gebruiken; een tekening op papier of een simpele tool als draw.io of Mermaid is prima.

Reflecteer: hoe verhoudt het relationele model in je database zich tot het objectmodel in Java? Waar zijn de mismatches?

Success criteria:

- Je hebt een UML-diagram met alle entiteiten en hun relaties
- Multipliciteiten zijn correct aangegeven
- Overerving (`Person` → `Student/Teacher`) is zichtbaar in het diagram
- Je kunt drie concrete voorbeelden noemen van object-relational mismatch in je project

Bonus Challenge (Optional)

Implementeer een lazy loading mechanisme met het decorator pattern. In plaats van gerelateerde entiteiten direct te laden, maak een proxy-object aan dat pas de database aanroept wanneer de data daadwerkelijk nodig is. Denk na over: wanneer is lazy loading voordelig? Wanneer levert het juist problemen op (hint: denk aan de sessie-context en het "detached entity" probleem)?

Bonus Challenge (Optional)

Schrijf een test waarbij je met behulp van heel veel (gefabriceerde) data de voordelen van geoptimaliseerde fetch strategieën aantoont.

Middag 5: Performance & Caching

Goal

Hoe performant is je framework eigenlijk? Vandaag heb je geleerd over caching op verschillende niveaus en performance-valkuilen. Nu ga je die kennis toepassen op je eigen framework.

Eerst ga je meten hoe je framework presteert, de bottlenecks opsporen en optimalisaties toevoegen.

Learning Goals

- Performance van het eigen framework meten door query-tellingen en executietijden te loggen
- Veelvoorkomende ORM-performance problemen (N+1 queries, overfetching) herkennen en oplossen in eigen code
- Een first-level cache (identity map) implementeren die herhaalde lookups van dezelfde entiteit voorkomt
- Evalueren wanneer caching helpt (en wanneer het het juist erger maakt)

Project Afternoon Parts

Het project van deze middag bevat **3 delen**.

Part 1: Performance Meten

Voordat je iets gaat optimaliseren, moet je weten waar je staat. Voeg instrumentatie toe aan je framework: tel het aantal queries dat wordt uitgevoerd en meet de tijd die ze kosten.

Bouw een [QueryLogger](#) of vergelijkbaar mechanisme dat:

- Elk SQL-statement logt dat je framework uitvoert
- Het totaal aantal queries per operatie bijhoudt
- De executietijd per query meet

Voer vervolgens een reeks operaties uit: haal alle [Departments](#) op met hun [Teachers](#) en bijbehorende [Courses](#). Hoeveel queries worden er uitgevoerd? Hoeveel zou je er verwachten?

Success criteria:

- Elke SQL-query die je framework uitvoert wordt gelogd met de SQL-tekst en executietijd
- Je kunt het totaal aantal queries per use case rapporteren
- Je hebt het N+1 probleem geïdentificeerd (of kunt uitleggen waarom het niet optreedt in jouw implementatie)
- Je hebt een baseline performance-meting voor het ophalen van departments met teachers en courses

Part 2: Optimisaties

Op basis van je metingen uit Part 1, implementeer tenminste twee optimalisaties:

1. **Query optimalisatie:** als je N+1 queries hebt gevonden, los dit op door JOIN queries te gebruiken of batch fetching toe te passen
2. **Index advies:** analyseer je queries en bepaal welke indexes je zou toevoegen. Voeg ze toe aan je schema en meet het verschil

Vergelijk de performance voor en na je optimalisaties. Gebruik concrete cijfers.

Success criteria:

- Ten minste twee optimalisaties zijn geïmplementeerd
- Performance voor en na is gemeten en gedocumenteerd met concrete cijfers
- De optimalisaties zijn uitgelegd: waarom helpen ze? In welke situaties zouden ze niet helpen?
- Het aantal queries voor het department/teacher/course scenario is afgenomen (of je kunt uitleggen waarom het al optimaal was)

Part 3: First-Level Cache (Identity Map)

Implementeer een first-level cache in je framework. Dit is een in-memory map die bijhoudt welke entiteiten al zijn opgehaald binnen de huidige "sessie" (of transactie). Als dezelfde entiteit opnieuw wordt opgevraagd, wordt deze uit de cache geserveerd in plaats van opnieuw uit de database gehaald.

Belangrijke overwegingen:

- Wanneer wordt de cache geleegd? (bij commit? bij rollback? handmatig?)
- Wat gebeurt er als een entiteit wordt gewijzigd? Is de cache dan nog geldig?
- Hoe voorkom je dat de cache stale data serveert?

Success criteria:

- Bij het twee keer ophalen van dezelfde entiteit (by ID) wordt slechts één query uitgevoerd
- De cache wordt geleegd bij het einde van een transactie
- Een test bewijst dat de cache werkt: query count is lager bij herhaalde lookups
- Een test bewijst dat na een update de cache correct wordt bijgewerkt of geïnvalideerd
- Je kunt uitleggen wanneer deze cache problematisch wordt (denk aan multi-threaded scenario's)

Bonus Challenge (Optional)

Implementeer een simpele second-level cache die entiteiten deelt tussen transacties. Gebruik een [ConcurrentHashMap](#) met een time-to-live (TTL) mechanisme. Meet de impact op performance bij read-heavy workloads versus write-heavy workloads. Wanneer is een second-level cache een goed idee, en wanneer is het vragen om problemen?

Wrap up (middag 6): Afronding & Presentatie

Goal

Dit is de laatste middag. Je gaat je framework loslaten op een set gegeven entiteiten, je domeinmodel vastleggen in UML, en je werk presenteerbaar maken.

Learning Goals

- Het eigen ORM framework toepassen op een nieuwe set entiteiten om de generaliseerbaarheid te valideren
- Een UML-diagram produceren dat het volledige domeinmodel documenteert inclusief relaties en overerving
- Kritisch reflecteren op de architectuur van het eigen framework en verbeterpunten identificeren

Project Afternoon Parts

Het project van deze middag bevat **3 delen**.

Part 1: Framework Toepassen op Nieuwe Entiteiten

Je maakt een set entiteiten die je nog niet in je framework hebt gebruikt. Definieer ze, annoteer ze, en gebruik je `GenericRepository` om ze op te slaan en op te halen.

Voorgestelde entiteiten (of gebruik een set die de trainer aanlevert):

- `Classroom` (id, roomNumber, capacity, building)
- `Schedule` (id, course, classroom, dayOfWeek, startTime, endTime)
- `Enrollment` (id, student, course, enrollmentDate, grade)

Zorg dat de relaties kloppen: een `Schedule` verwijst naar een `Course` en een `Classroom`. Een `Enrollment` koppelt een `Student` aan een `Course`.

Success criteria:

- Alle nieuwe entiteiten zijn geannoteerd en werken met je framework
- CRUD-operaties werken voor alle nieuwe entiteiten
- Relaties worden correct opgehaald (eager fetching)
- Je framework had weinig tot geen aanpassingen nodig om de nieuwe entiteiten te ondersteunen

Part 2: UML Diagram

Maak een volledig UML-klassediagram van je complete domeinmodel. Dit diagram bevat alle entiteiten die je in de loop van het project hebt gebouwd: `Student`, `Teacher`, `Person`, `Course`, `Department`, `Classroom`, `Schedule`, `Enrollment`.

Neem op:

- Alle attributen per entiteit
- Alle relaties met multipliciteit
- Overerving waar van toepassing
- Primary keys en foreign keys

Dit diagram wordt onderdeel van je presentatie.

Success criteria:

- Het UML-diagram bevat alle entiteiten met attributen
- Relaties zijn correct weergegeven met multipliciteit
- Overerving is zichtbaar
- Het diagram is leesbaar en goed gestructureerd

Part 3: Reflectie en Presentatie

Bereid een korte presentatie voor (10-15 minuten per groep) waarin je het volgende behandelt:

1. **Architectuur:** hoe is je framework opgebouwd? Welke lagen heb je? Toon een kort overzicht.
2. **UML-diagram:** presenteer je voorbeeld domeinmodel en leg de belangrijkste relaties uit.
3. **Toepassing:** laat je framework in actie zien met je voorbeeld domeinmodel.
4. **Wat ging goed:** welke ontwerpkeuzes ben je blij mee? Waar werkte je abstractie goed?
5. **Wat was lastig:** waar liep je tegen de grenzen van je framework aan? Wat zou je anders doen?
6. **Vergelijking met Hibernate:** nu je weet hoe een ORM werkt onder de motorkap, wat waardeer je meer (of minder) aan Hibernate?

Bonus Challenge (Optional)

Schrijf een "developer guide" voor je framework: een kort document (max 2 pagina's) dat een nieuwe developer uitlegt hoe ze een entiteit toevoegen aan het framework. Inclusief: kleine voorbeelden, welke annotaties nodig zijn, hoe relaties werken, en welke beperkingen er zijn.